Jet Set Willy evolutiva — Fix delle rampe al cambio stanza

Data: 12 agosto 2026 · File toccati: **physics.asm**, **input.asm**, **data/datavariable.asm** · Build: sjasmplus, 0 errori

Sintomi. (1) Salendo da una stanza a quella superiore, Willy veniva “calamitato” dalla rampa della stanza di arrivo anche se non vi era sopra (es. First Landing → Top Landing: appariva in cima alle scale invece che in basso). (2) Scendendo una rampa che continua nella stanza inferiore (Conservatory Roof → Orangery), Willy ricompariva a mezz’aria spostato in salita e cadeva, invece di proseguire la discesa sulla rampa.

Entrambe le cause sono state individuate e verificate riproducendo gli scenari in un’emulazione Z80 headless della .sna (script in **scratch/**, vedi in fondo).

Fix 1 — Calamita della rampa salendo di stanza

Causa principale. In **physics.asm**, nel controllo di atterraggio su rampa (**.falling_checks** di **Phys_ApplyGravity**), la Y precedente di Willy (old_Y) veniva tenuta nel registro **D** attraverso la chiamata a **GetRampY** — ma GetRampY usa D come registro di lavoro e ci lascia la Y calcolata della rampa. Il confronto “old_Y > ramp_Y? allora Willy era sotto la rampa, non atterrare” confrontava quindi ramp_Y con se stessa: risultava sempre “uguale” e l’atterraggio da sotto era sempre permesso. Al cambio stanza l’altra guardia (jump_start_y) viene azzerata da TriggerRoomChange, quindi il primo frame di caduta nella stanza nuova agganciava la rampa a qualunque distanza (teleport di 58 px nel caso del video).

1a) physics.asm — .jump_start_ok (correzione decisiva)

PRIMA:

```
.jump_start_ok:
    ld   a, d               ; A = old_Y   <-- D gia' clobberato da GetRampY!
    cp   h
    jr   z, .check_new_y
    jp   nc, .no_ramp_landing ; Se old_Y > ramp_Y (era sotto la rampa), non atterra
```

DOPO:

```
.jump_start_ok:
    ; NOTA: D conteneva old_Y ma GetRampY lo clobbera (ld d,a con la Y calcolata),
    ; quindi il confronto D vs H era sempre "uguale" e Willy atterrava sulla
    ; rampa anche arrivando da sotto. Ricarica old_Y dalla memoria.
    ld   a, (PlayerPos+1)   ; A = old_Y
    cp   h
    jr   z, .check_new_y
    jp   nc, .no_ramp_landing ; Se old_Y > ramp_Y (era sotto la rampa), non atterra
```

1b) data/datavariable.asm — nuova variabile (riga 68)

```
room_change_grounded: db 0 ; on_ground al momento del cambio stanza (per l'aggancio rampa)
```

1c) input.asm — TriggerRoomChange: salvataggio di on_ground prima del reset (riga ~548)

```
    ld   (room_ptr), de            ; Aggiorna room_ptr con la nuova stanza prima delle verifiche rampa!

    ; AGGIUNTO:
    ; Ricorda se Willy era a terra: entrando dal basso l'aggancio alla rampa
    ; vale solo se ci stava camminando sopra (rampa che continua tra le stanze)
    ld   a, (on_ground)
    ld   (room_change_grounded), a

    ; Reset completo della fisica del player per la nuova stanza
    xor  a
    ...
```

1d) input.asm — TriggerRoomChange: gate sull'aggancio entrando dal basso (riga ~571)

PRIMA:

```
    ; Transizione verticale!
    xor  a
    ld   (ramp_entry_guard), a
    call Phys_CheckRampOnRoomChange ; Se allineato, modifica B/C e imposta on_ground = 1
    jr   .ramp_guard_done
```

DOPO:

```
    ; Transizione verticale!
    xor  a
    ld   (ramp_entry_guard), a
    ; Ingresso dal basso (B nella meta' inferiore): aggancia la rampa solo se
    ; Willy ci stava camminando sopra. Se arriva in salto, niente calamita:
    ; sara' la fisica normale a farlo atterrare sulla rampa se ci ricade sopra.
    ld   a, b
    cp   48
    jr   c, .vert_ramp_check       ; Ingresso dall'alto (B=4): comportamento invariato
    ld   a, (room_change_grounded)
    or   a
    jr   z, .ramp_guard_done       ; Era in volo: nessun aggancio alla rampa
.vert_ramp_check:
    call Phys_CheckRampOnRoomChange ; Se allineato, modifica B/C e imposta on_ground = 1
    jr   .ramp_guard_done
```

Nota: 1b–1d sono una protezione aggiuntiva (evitano lo snap di 16 px a chi entra saltando); la causa vera del sintomo era 1a.

Fix 2 — Caduta a vuoto scendendo una rampa che continua nella stanza sotto

Causa. In **Phys_CheckRampOnRoomChange** (physics.asm) il controllo dei bordi eseguiva **cp 104** (bordo inferiore) prima di **bit 7** (segno). Quando la rampa della stanza di arrivo prosegue oltre la cima dello schermo, GetRampY restituisce una Y negativa (es. –2 = 254 unsigned): il confronto unsigned la scambiava per “bordo inferiore” e **.adj_bottom** spostava X *in salita* di 8 in 8 fino a uscire dal range della rampa — risultato: nessun aggancio e Willy piazzato ~25 px piu' a destra, a mezz’aria. Con il fix il valore negativo va in **.adj_top**, che scende lungo la pendenza fino a una Y visibile e aggancia correttamente.

2a) physics.asm — controllo bordi all'ingresso (riga ~1425)

PRIMA:

```
    ; Controllo bordo inferiore (ramp_Y >= 104)
    ld   a, e              ; A = ramp_Y
    cp   104
    jr   nc, .adj_bottom   ; <-- un ramp_Y negativo (128..255) finiva qui!

    ; Controllo bordo superiore (ramp_Y <= 4 o negativo)
    bit  7, a
    jr   nz, .adj_top
    cp   5
    jr   c, .adj_top
    jr   .apply_ramp_align
```

DOPO:

```
    ; Controllo bordo superiore (ramp_Y negativo = rampa oltre la cima):
    ; va verificato PRIMA del bordo inferiore, perche' in unsigned un valore
    ; negativo (128..255) supera anche il confronto con 104 e finirebbe in
    ; .adj_bottom spostando X in salita fuori dalla rampa.
    ld   a, e              ; A = ramp_Y
    bit  7, a
    jr   nz, .adj_top

    ; Controllo bordo inferiore (ramp_Y >= 104)
    cp   104
    jr   nc, .adj_bottom
    cp   5
    jr   c, .adj_top
    jr   .apply_ramp_align
```

2b) physics.asm — stesso riordino nel loop .recalc_ramp_y (riga ~1483)

PRIMA:

```
    ld   e, a              ; E = nuovo ramp_Y safe

    ; Controlla se ancora ai bordi
    cp   104
    jr   nc, .adj_bottom
    bit  7, a
    jr   nz, .adj_top
    cp   5
    jr   c, .adj_top
```

DOPO:

```
ld   e, a              ; E = nuovo ramp_Y safe

; Controlla se ancora ai bordi (il segno PRIMA del confronto con 104,
; come sopra: un ramp_Y negativo deve andare in .adj_top)
bit  7, a
jr   nz, .adj_top
cp   104
jr   nc, .adj_bottom
cp   5
jr   c, .adj_top
```

Verifica (emulazione headless della .sna)

Scenario	Prima del fix	Dopo il fix
Salita First Landing → Top Landing (camminando su rampa, esce dal soffitto)	Entra a (170,96); al 1° frame di caduta teleport a (170,38) sulla rampa, poi risale fino in cima	Entra a (170,96), cade e atterra sul pavimento a Y=104: arriva “in basso” come atteso
Discesa Conservatory Roof → Orangery (rampa che continua tra le stanze)	Entra a (95,4); spostato a X≈119 fuori rampa, nessun aggancio, caduta a vuoto	Entra a (95,4); agganciato in discesa a (79,9) e prosegue: (76,12) (73,15) (70,18)…
Salita su rampa nella stessa stanza	OK	OK (invariato)
Passaggio sotto una rampa camminando	A rischio falso atterraggio	OK

Strumenti di test lasciati nel progetto

scratch/jsw_headless_trace.py — scenario salita FL→TL: emula la .sna senza emulatore grafico (pip install z80 + ROM 48.rom di Fuse), posiziona Willy alla base della rampa, tiene premuto “destra” e traccia posizione, fisica e transizioni frame per frame con breakpoint su TriggerRoomChange.

scratch/jsw_headless_descend.py — scenario discesa CR→Orangery, stessa tecnica con tasto “sinistra”.

Promemoria per il futuro: GetRampY non preserva D/E/H/L — mai tenere valori nei registri attraverso quella call; le Y di rampa possono essere negative — testare bit 7 prima di ogni confronto unsigned.